3053994_BIOL5177_assignment2

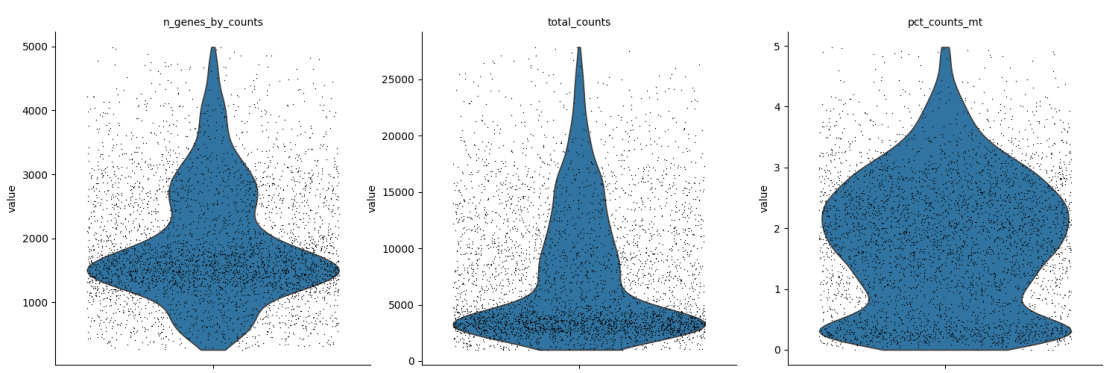
April 9, 2025

[1]:
```
# Import libraries
import scanpy as sc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import anndata as ad
import seaborn as sns
import harmonypy as hm
from pydeseq2.dds import DeseqDataSet, DefaultInference
from pydeseq2.ds import DeseqStats
import numba as nb
import phate
from upsetplot import UpSet, from_memberships
import matplotlib.pyplot as plt
```

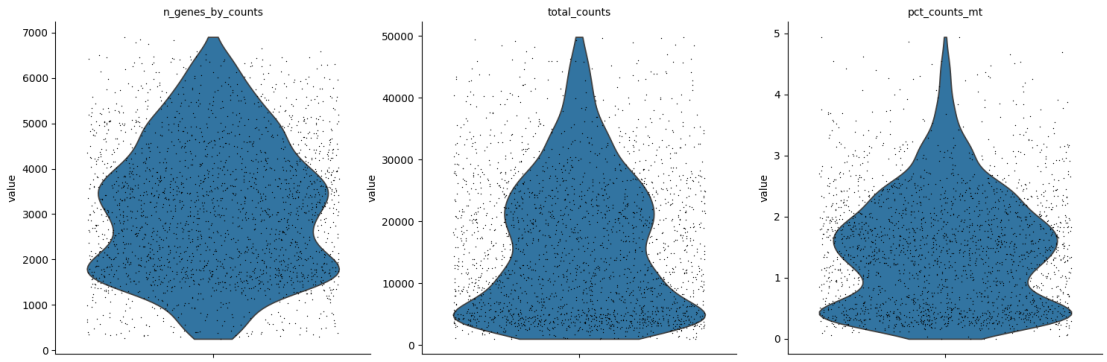
[2]:
```
# My functions (for pre-processing pipeline)

# load sample data into the script and create AnnData object for each sample␣
↪and add batch labels (WT1, INF1, INF2)into the AnnData Object
def load_data(counts_path, metadata_path, batch_label):
    counts = pd.read_csv(counts_path, index_col=0).T
    metadata = pd.read_csv(metadata_path, index_col=0).loc[counts.index]
    adata = sc.AnnData(X=counts, obs=metadata)
    adata.obs['batch'] = batch_label
    return adata

# Identify mitochondrial genes + Calculate the percentage of counts in␣
↪mitochondrial genes per cell
def add_mito_qc(adata, mito_prefix='mt-'):
    adata.var['mt'] = adata.var_names.str.startswith(mito_prefix)
    sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'], percent_top=None,␣
↪log1p=False, inplace=True)
    return adata

# filter out doublets and empty droplets and dying cells
def filter_cells(adata, min_genes=100, max_genes=3500, max_mito=4):
    return adata[
```

```
        (adata.obs['n_genes_by_counts'] > min_genes) &
        (adata.obs['n_genes_by_counts'] < max_genes) &
        (adata.obs['pct_counts_mt'] < max_mito),
        :
    ].copy()

# show violin plots to decide the cut off values for min_genes, max_genes,␣
↪max_mito in filter_cells function above
def plot_qc_violins(adata, title=''):
    sc.pl.violin(
        adata,
        ['n_genes_by_counts', 'total_counts', 'pct_counts_mt'],
        jitter=0.4,
        multi_panel=True
    )
    plt.suptitle(title)  # Use the 'title' parameter

# the single function that will be used in main script to apply all the␣
↪functions above for pre-processing pipeline
def process_sample(counts_path, metadata_path, batch_label,
                   min_genes=100, max_genes=3500, max_mito=4, plot=True):

    adata = load_data(counts_path, metadata_path, batch_label)
    add_mito_qc(adata)

    if plot:
        plot_qc_violins(adata, title=batch_label)

    adata = filter_cells(adata, min_genes=min_genes, max_genes=max_genes,␣
↪max_mito=max_mito)
    return adata

#-----------------------------------------------------------------------
#function for making pseudo replicate for pseudo bulk analysis
#only have 1 sample for WT so not enough power to do pseudo bulk DE analysis
#so create pseudo replicates within WT and Infected groups (3 replicates in␣
↪each group)
def assign_pseudoreps(adata, condition_col='condition', n_reps=3):
    adata.obs['pseudorep'] = None
    for cond in adata.obs[condition_col].unique():
        cond_cells = adata.obs_names[adata.obs[condition_col] == cond]
        shuffled = np.random.permutation(cond_cells)
        split = np.array_split(shuffled, n_reps)
        for i, chunk in enumerate(split):
            adata.obs.loc[chunk, 'pseudorep'] = f"{cond}_rep{i+1}"
    return adata
```

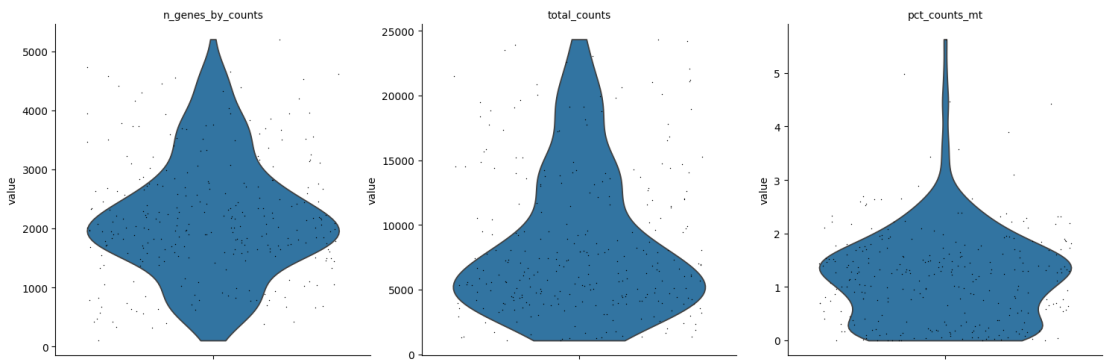
[3]:
```
# Main script
# Pre-processing pipeline to make AnnData objects (see functions box above for␣
↪the details of pre-processing)
adata_wt1 = process_sample(
    '/data/BIOL5177/Assessment/WT1/counts_matrix.csv',
    '/data/BIOL5177/Assessment/WT1/metadata.csv',
    'WT1',
    min_genes=500,
    max_genes=3000,
    max_mito=5
)

adata_inf1 = process_sample(
    '/data/BIOL5177/Assessment/Infected1/counts_matrix.csv',
    '/data/BIOL5177/Assessment/Infected1/metadata.csv',
    'Infected1',
    min_genes=1000,
    max_genes=4000,
    max_mito=5
)

adata_inf2 = process_sample(
    '/data/BIOL5177/Assessment/Infected2/counts_matrix.csv',
    '/data/BIOL5177/Assessment/Infected2/metadata.csv',
    'Infected2',
    min_genes=400,
    max_genes=3000,
    max_mito=5
)
```




```
<Figure size 640x480 with 0 Axes>
```




```
<Figure size 640x480 with 0 Axes>
```

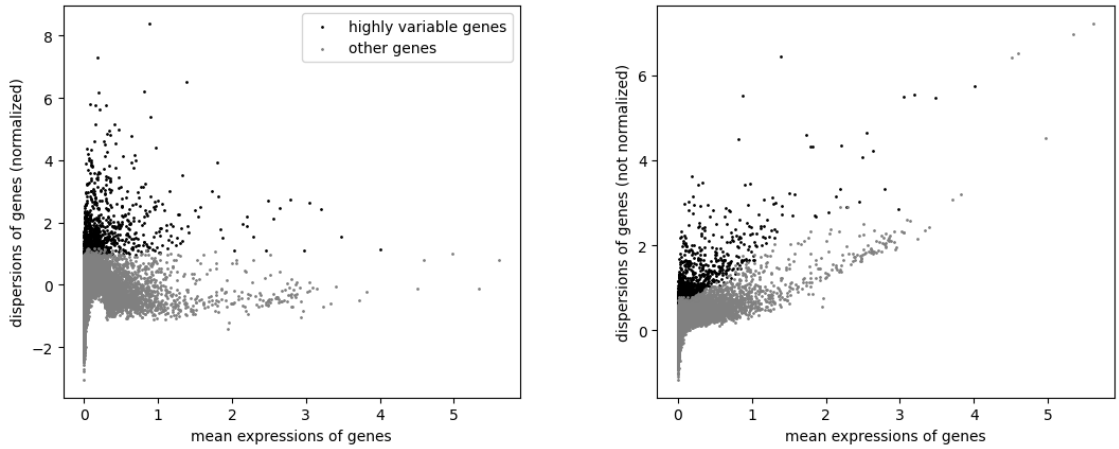



```
<Figure size 640x480 with 0 Axes>
```

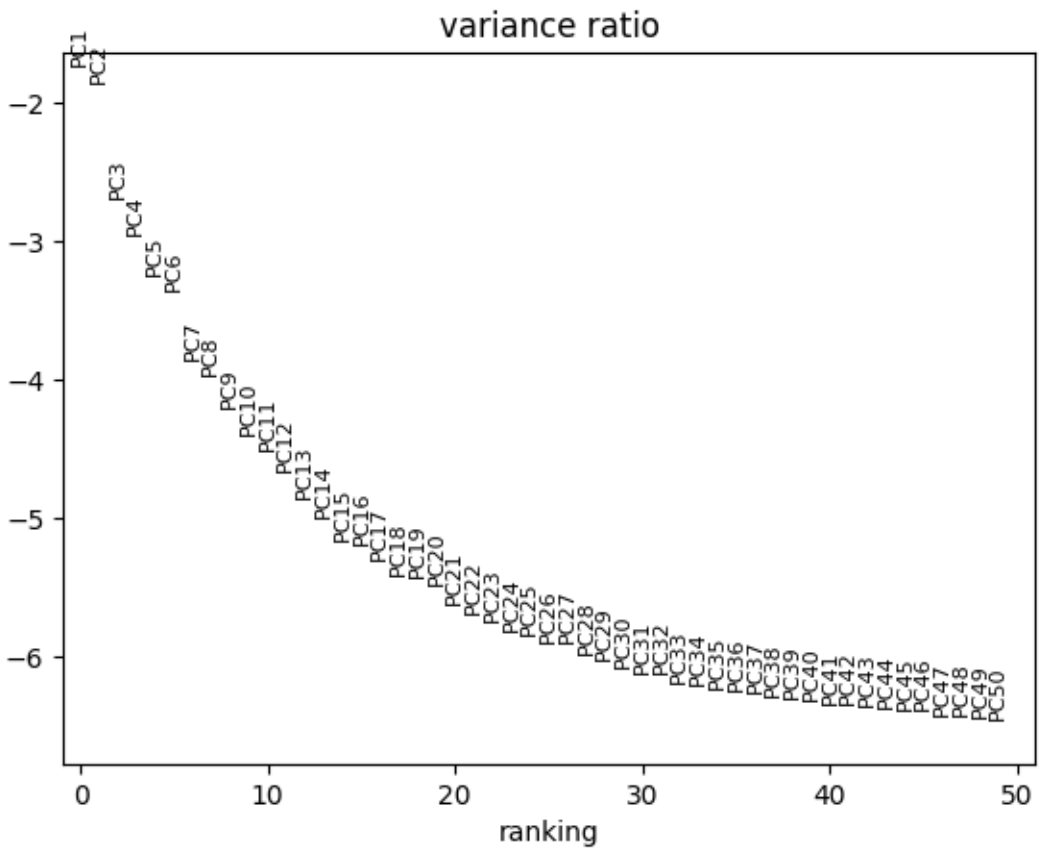
[4]:
```
# Concatenate into one AnnData object
# Put all adatas in a list
adatas = [adata_wt1, adata_inf1, adata_inf2]

# Concatenate them into a single AnnData
adata_concat = ad.concat(adatas, label='batch', keys=['WT1', 'Infected1',␣
↪'Infected2'], join='outer')
```

[5]:
```
# Normalizing to median total counts
sc.pp.normalize_total(adata_concat)
# Logarithmize the data
sc.pp.log1p(adata_concat)
```

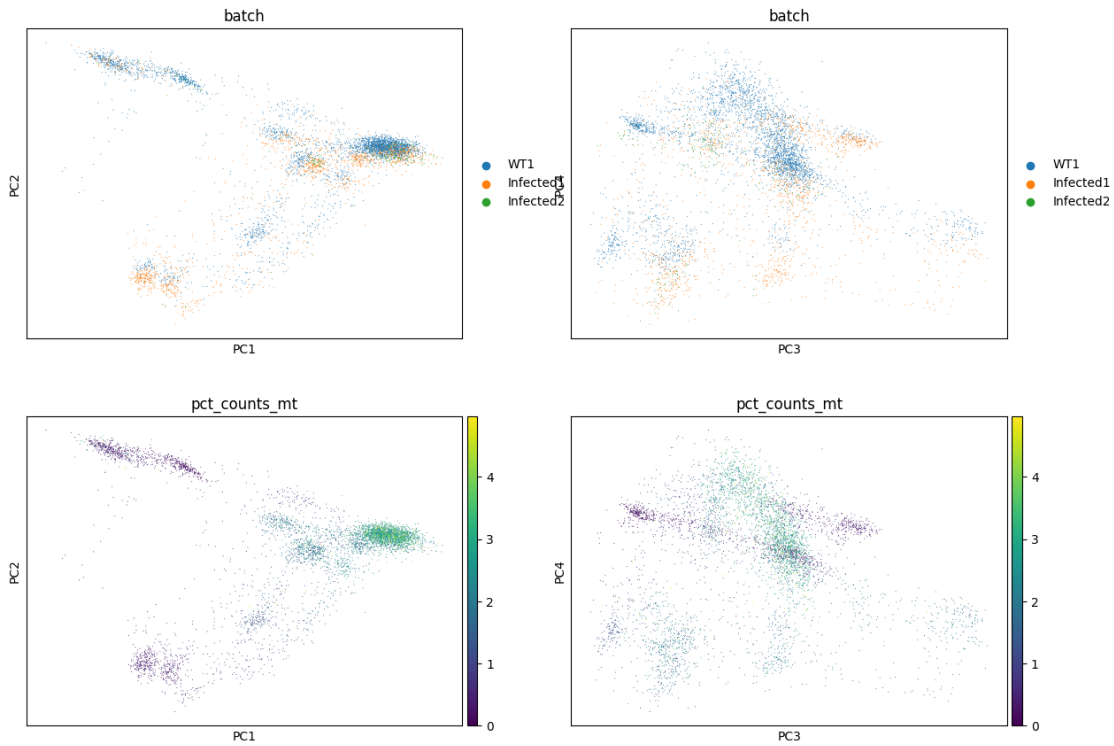
[6]:
```
# Feature selection
sc.pp.highly_variable_genes(adata_concat, n_top_genes=1000, batch_key="batch")
sc.pl.highly_variable_genes(adata_concat)
```



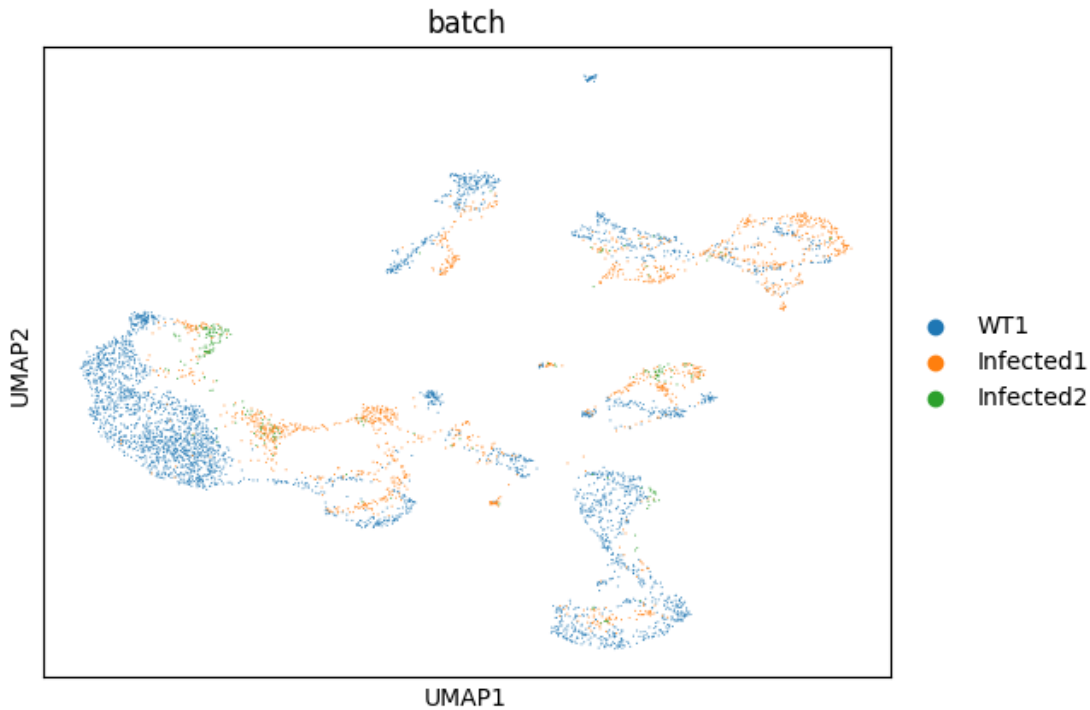
[7]:
```
# Dimention reduction PCA
sc.tl.pca(adata_concat)
sc.pl.pca_variance_ratio(adata_concat, n_pcs=50, log=True)
```



[8]:
```
# PCA plots
sc.pl.pca(
    adata_concat,
    color=["batch", "batch", "pct_counts_mt", "pct_counts_mt"],
    dimensions=[(0, 1), (2, 3), (0, 1), (2, 3)],
    ncols=2,
    size=2,
)
```

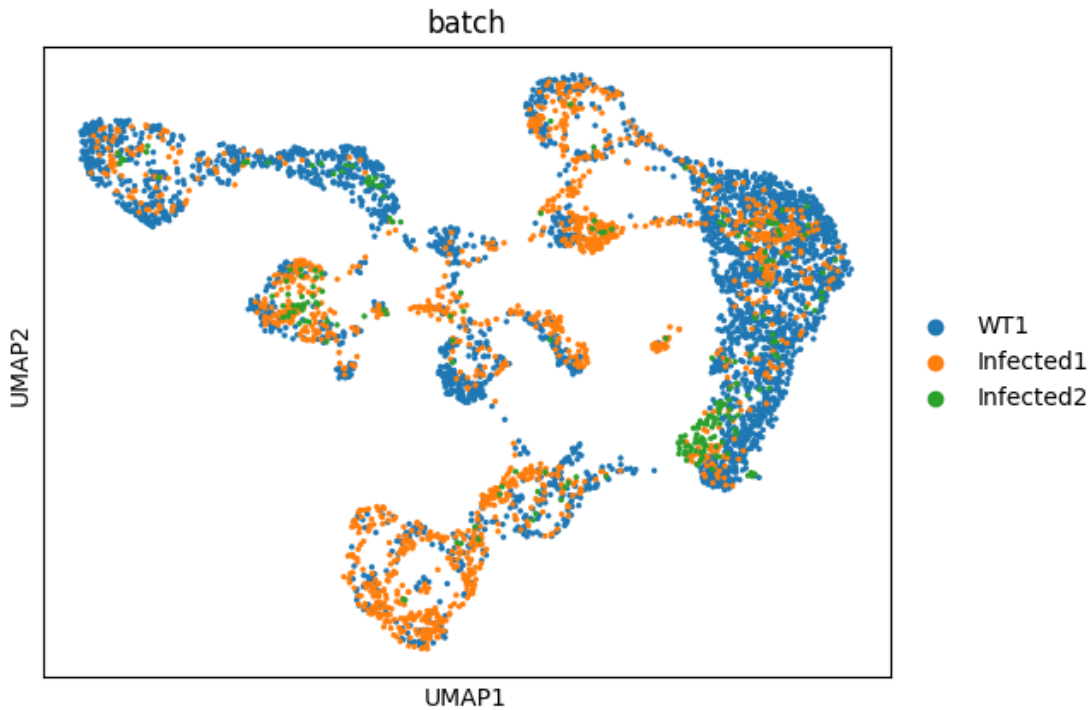


[9]:
```
# Nearest neighbor graph constuction and visualization
sc.pp.neighbors(adata_concat)
sc.tl.umap(adata_concat)
sc.pl.umap(
    adata_concat,
    color="batch",
    # Setting a smaller point size to get prevent overlap
    size=2,
)
```

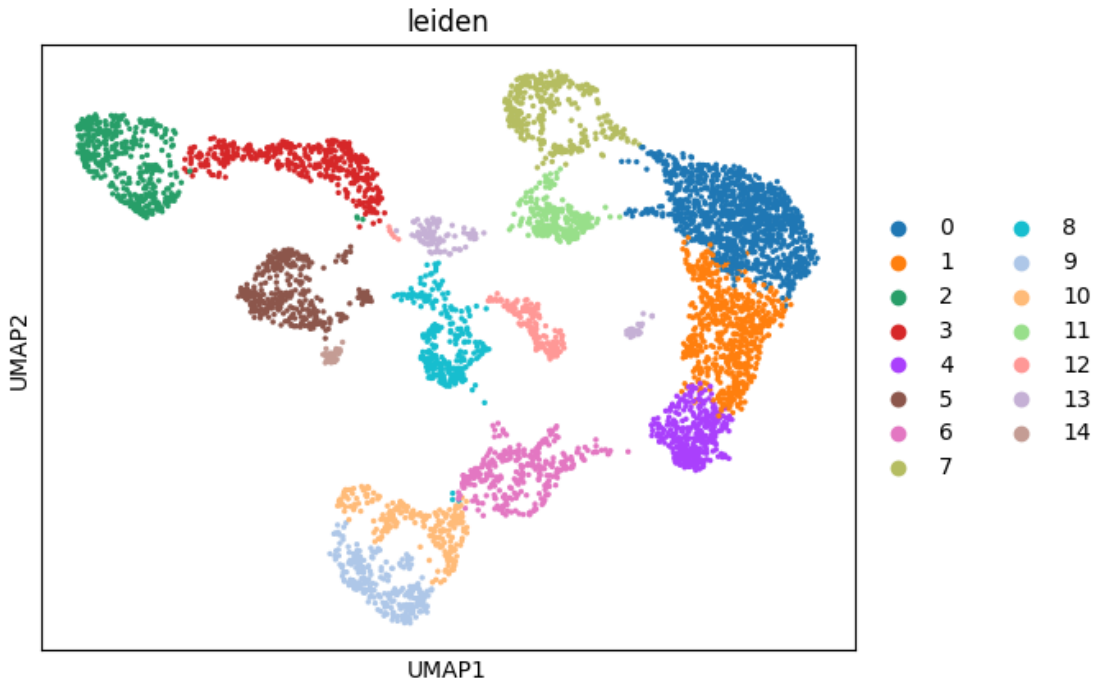


[10]:
```
# BKNN integration
sc.tl.pca(adata_concat)
sc.external.pp.bbknn(adata_concat, batch_key="batch")  # running bbknn 1.3.6
sc.tl.umap(adata_concat)
sc.pl.umap(adata_concat, color=["batch"]) #UMAP to show samples integrated
sc.tl.leiden(adata_concat, resolution=0.5)
```

```
WARNING: consider updating your call to make use of `computation`
```

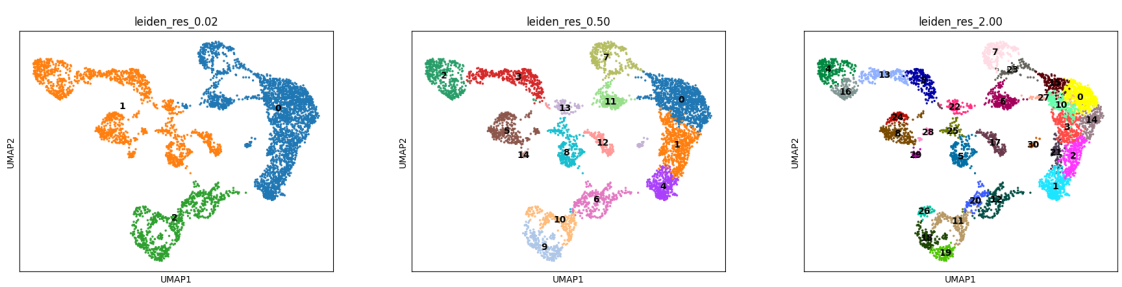


[11]:
```
# Using the igraph implementation and a fixed number of iterations can be
 ↪significantly faster, especially for larger datasets
#sc.tl.leiden(adata_concat, n_iterations=2)
sc.pl.umap(adata_concat, color=["leiden"]) #UMAP to show clusters
```



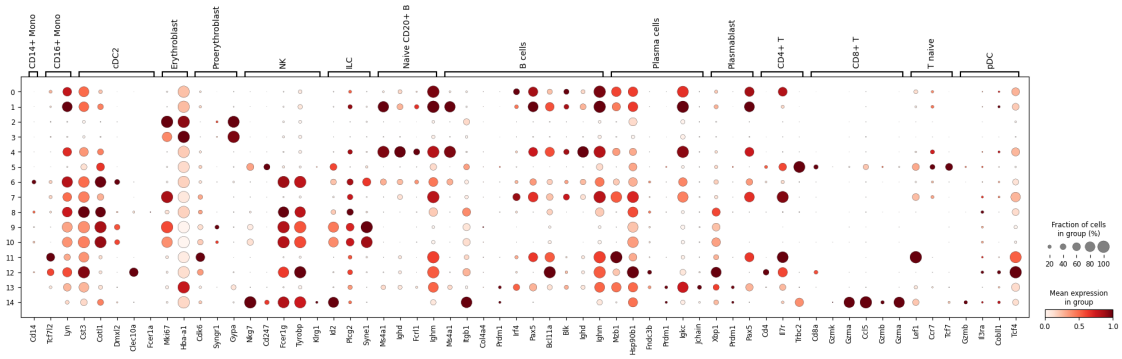
[12]:
```
for res in [0.02, 0.5, 2.0]:
    sc.tl.leiden(
        adata_concat, key_added=f"leiden_res_{res:4.2f}", resolution=res
    )
```

[13]:
```
sc.pl.umap(
    adata_concat,
    color=["leiden_res_0.02", "leiden_res_0.50", "leiden_res_2.00"],
    legend_loc="on data",
)
```



[14]:
```
# Visualise what genes are represented in all 14 different clusters
```

```python
marker_genes = {
    "CD14+ Mono": ["Cd14"],
    "CD16+ Mono": ["Tcf7l2", "Lyn"],
    # Note: Dmxl2 should be negative
    "cDC2": ["Cst3", "Cotl1", "Dmxl2", "Clec10a", "Fcer1a"],
    "Erythroblast": ["Mki67", "Hba-a1"],
    # Note Hbm and Gypa are negative markers
    "Proerythroblast": ["Cdk6", "Syngr1", "Gypa"],
    "NK": ["Nkg7", "Cd247", "Fcer1g", "Tyrobp", "Klrg1"],
    "ILC": ["Id2", "Plcg2", "Syne1"],
    "Naive CD20+ B": ["Ms4a1", "Ighd", "Fcrl1", "Ighm"],
    # Note Ighd and Ighm are negative markers
    "B cells": [
        "Ms4a1",
        "Itgb1",
        "Col4a4",
        "Prdm1",
        "Irf4",
        "Pax5",
        "Bcl11a",
        "Blk",
        "Ighd",
        "Ighm",
    ],
    "Plasma cells": ["Mzb1", "Hsp90b1", "Fndc3b", "Prdm1", "Igkc", "Jchain"],
    # Note Pax5 is a negative marker
    "Plasmablast": ["Xbp1", "Prdm1", "Pax5"],
    "CD4+ T": ["Cd4", "Il7r", "Trbc2"],
    "CD8+ T": ["Cd8a", "Gzmk", "Gzma", "Ccl5", "Gzmb", "Gzma"],
    "T naive": ["Lef1", "Ccr7", "Tcf7"],
    "pDC": ["Gzmb", "Il3ra", "Cobll1", "Tcf4"],
}

sc.pl.dotplot(adata_concat, marker_genes, groupby="leiden_res_0.50",␣
 ↪standard_scale="var")
```

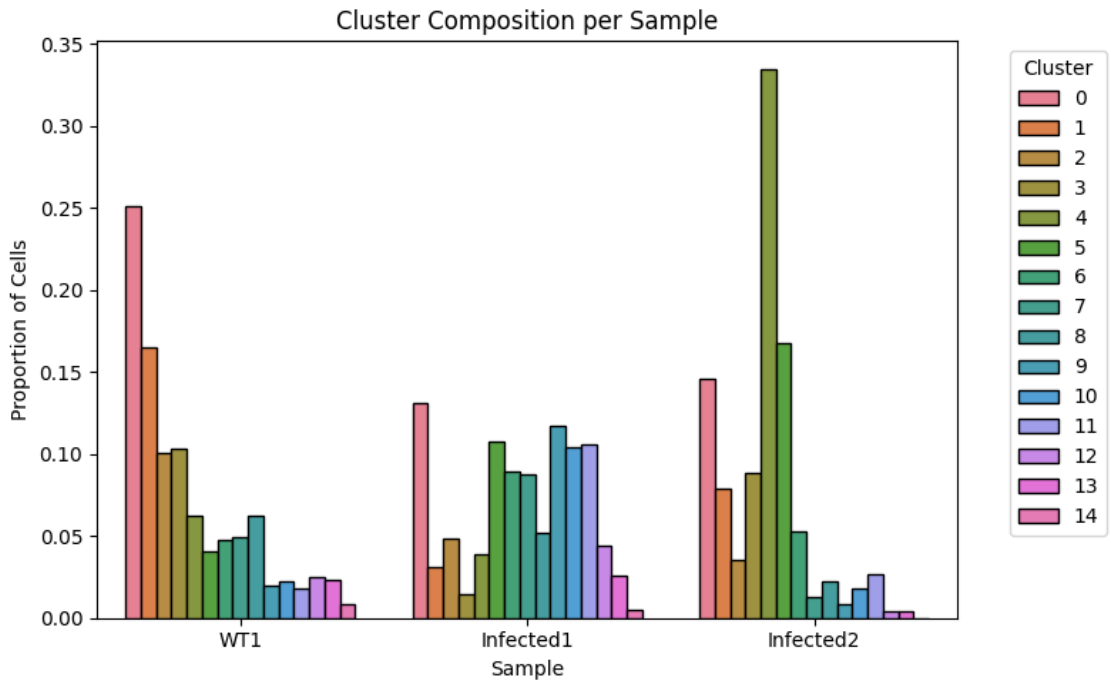


[15]:
```
# Count cells per Leiden cluster to identify top 3 largest clusters
cluster_sizes = adata_concat.obs['leiden'].value_counts().
 ↪sort_values(ascending=False)
print(cluster_sizes.head(3))
```

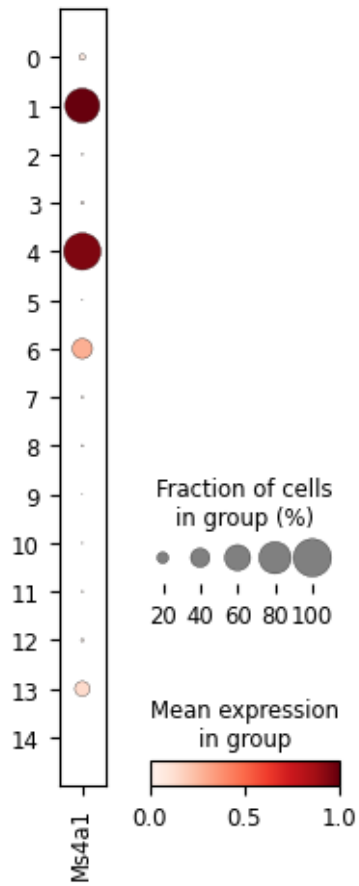
```
leiden
0    1079
1     625
2     422
Name: count, dtype: int64
```

[16]:
```
# Annotate the top 3 largest clusters
adata_concat.obs["cell_type_lvl1"] = adata_concat.obs["leiden_res_0.50"].map(
    {
        "0": "B cells",
        "1": "Naive CD20+ B",
        "2": "Erythroblast",
    }
)
```

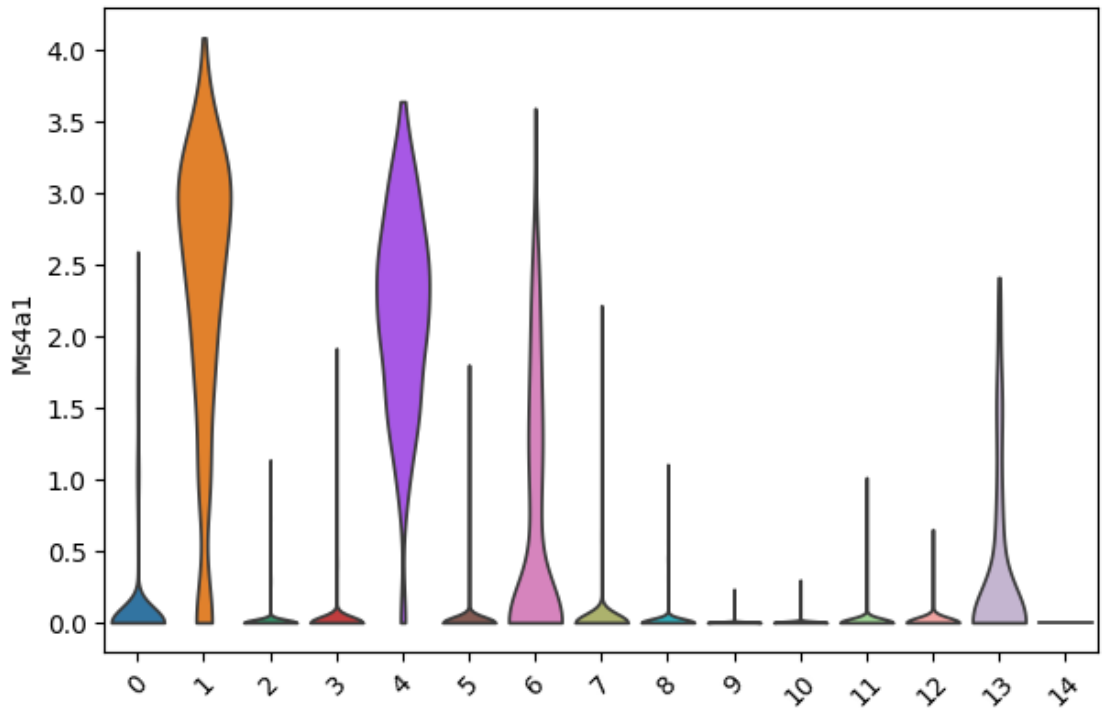
[17]:
```
# Plot the frequency of cell types in the clusters to allow frequency comparison
# Create a frequency table: clusters × samples
freq_table = pd.crosstab(adata_concat.obs['leiden'], adata_concat.obs['batch'])

# Convert counts to proportions (column-wise)
freq_table_prop = freq_table.div(freq_table.sum(axis=0), axis=1)

# Convert to long format for seaborn
freq_table_long = freq_table_prop.reset_index().melt(id_vars='leiden',␣
 ↪var_name='Sample', value_name='Proportion')

# Plot
plt.figure(figsize=(8, 5))
sns.barplot(data=freq_table_long, x='Sample', y='Proportion', hue='leiden',␣
 ↪edgecolor='black')

plt.xlabel("Sample")
plt.ylabel("Proportion of Cells")
plt.title("Cluster Composition per Sample")
plt.legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



[18]:
```
# Check which cluster expresses Ms4a1
sc.pl.dotplot(adata_concat, ['Ms4a1'], groupby='leiden', standard_scale='var')
```



[19]:
```
# Use violin plot to see cluster 1 or 4 has higher Ms4a1
sc.pl.violin(
    adata_concat,
    keys=["Ms4a1"],
    groupby="leiden_res_0.50",
    stripplot=False,
    rotation=45,
    multi_panel=True
)
# Cluster 1 has higher Ms4a1 so cluster 1 will be chosen for downstream analysis
```



[20]:
```
# DE analysis for BKNN integrated data
# Select cells from clusters 1 (Ms4a1 cluster)
adata_b_like = adata_concat[adata_concat.obs['leiden'].isin(['1'])].copy()
```

[21]:
```
# Create a unified condition column
adata_b_like.obs['condition'] = adata_b_like.obs['batch'].replace({
    'WT1': 'WT',
    'Infected1': 'Infected',
    'Infected2': 'Infected'
})
```

[22]:
```
# Perform differential expression within the selected clusters
sc.tl.rank_genes_groups(
    adata_b_like,
    groupby='condition',
    groups=['WT'],
    reference='Infected',
    method='wilcoxon'
)
```

[23]:
```
# Convert to DataFrame and preview
de_results = sc.get.rank_genes_groups_df(adata_b_like, group='WT')
print(de_results.head(20))
```

```
# Save to CSV just in case
de_results.to_csv("DE_WT_vs_Infected_in_Ms4a1_clusters_1_4.csv", index=False)
```

```
         names     scores  logfoldchanges         pvals     pvals_adj
0       Hbb-bs  12.248159        2.363914  1.718520e-34  3.646012e-30
1       Hbb-bt  12.072429        3.229774  1.477092e-33  1.566899e-29
2       Hba-a1  11.547064        2.301171  7.638534e-31  4.051479e-27
3       S100a9   9.775876        1.724076  1.429153e-22  2.332378e-19
4       Rps3a1   9.211785        0.776759  3.207461e-20  3.780528e-17
5       Hba-a2   8.269563        2.920841  1.344513e-16  1.141008e-13
6       S100a8   8.154479        1.482808  3.506875e-16  2.755625e-13
7        H3f3a   6.994146        0.582613  2.668785e-12  1.230890e-09
8      mt-Atp6   6.582769        0.840539  4.617658e-11  1.959365e-08
9        Rps19   5.922227        0.456424  3.176106e-09  1.036681e-06
10  Rpl21-ps15   5.714346        0.767927  1.101265e-08  3.157357e-06
11      Gm1821   5.674524        2.550175  1.390749e-08  3.840152e-06
12         Fau   5.567843        0.358404  2.579115e-08  7.015192e-06
13   Rps19-ps6   5.564920       29.070906  2.622718e-08  7.043491e-06
14        Fth1   5.429743        0.583301  5.643516e-08  1.437109e-05
15      mt-Co2   5.275568        1.182510  1.323454e-07  3.019183e-05
16     mt-Cytb   5.060015        0.519038  4.192232e-07  8.984080e-05
17       Actg1   4.790391        0.610045  1.664564e-06  3.363371e-04
18        Rps8   4.661790        0.357084  3.134708e-06  5.991528e-04
19        Ftl1   4.431623        0.701091  9.352628e-06  1.639879e-03
```

[25]:
```
# Get DE results comparing WT vs Infected (BKNN)
de_df = sc.get.rank_genes_groups_df(adata_b_like, group='WT')

# Filter for genes upregulated in Infected (logFC < -0.5 and adjusted p < 0.01)
up_in_infected_bknn = de_df[
    (de_df['logfoldchanges'] < -0.5) &
    (de_df['pvals_adj'] < 0.01)
]['names'].tolist()

# Optional: Preview number of genes
print(f"Genes upregulated in Infected (BKNN): {len(up_in_infected_bknn)}")
```

```
Genes upregulated in Infected (BKNN): 117
```

[26]:
```
# Pseudo bulk for BKNN integrated data

adata = adata_b_like.copy()
# Combine Infected1 and Infected2 into 'Infected'
adata.obs['condition'] = adata.obs['batch'].replace({
    'WT1': 'WT',
    'Infected1': 'Infected',
```

```
    'Infected2': 'Infected'
})
adata_ms4a1 = adata[adata.obs['leiden_res_0.50'] == '1'].copy()

# ----------------------------------------
# Call the function that makes pseudo replicates (3 per condition)
# Reason for making pseufo replicates explained in function kernel (see kernel␣
↪2/box2/cell2)
# ----------------------------------------

adata_ms4a1 = assign_pseudoreps(adata_ms4a1, condition_col='condition',␣
↪n_reps=3)

# ----------------------------------------
# Pseudo-bulk aggregation by pseudorep
# ----------------------------------------
expr_df = pd.DataFrame(
    adata_ms4a1.X.toarray() if hasattr(adata_ms4a1.X, "toarray") else␣
↪adata_ms4a1.X,
    index=adata_ms4a1.obs_names,
    columns=adata_ms4a1.var_names
)

# Concatenate metadata
combined = pd.concat([adata_ms4a1.obs[['pseudorep', 'condition']], expr_df],␣
↪axis=1)

# Aggregate counts by pseudoreplicate
bulk_df = combined.groupby('pseudorep')[expr_df.columns].sum()

# Round and convert to integer counts
counts = bulk_df.round().astype(int)

# Metadata for DESeq2
metadata = adata_ms4a1.obs[['pseudorep', 'condition']].drop_duplicates().
↪set_index('pseudorep')

# Create AnnData object
adata_pb = ad.AnnData(X=counts.values)
adata_pb.var_names = counts.columns
adata_pb.obs = metadata

# ----------------------------------------
# Run DESeq2
# ----------------------------------------
inference = DefaultInference(n_cpus=8)
```

```python
dds = DeseqDataSet(
    adata=adata_pb,
    design_factors='condition',
    ref_level=['condition', 'WT'],  # WT is reference group
    refit_cooks=True,
    inference=inference,
)
dds.deseq2()

stat_res = DeseqStats(
    dds,
    contrast=["condition", "WT", "Infected"],
    inference=inference,
)
stat_res.summary()

# Extract DE results
results_df = stat_res.results_df

# ----------------------------------------
# Filter for upregulated genes in infected cells
# ----------------------------------------
# log2FoldChange > 0.5 and adjusted p-value < 0.01
up_in_infected_pb_bknn_df = results_df[
    (results_df['padj'] < 0.01) &
    (results_df['log2FoldChange'] > 0.5)
]

# Extract gene names as a list
# Check if gene names are in index or column
if 'gene' in up_in_infected_pb_bknn_df.columns:
    up_in_infected_pb_bknn = up_in_infected_pb_bknn_df['gene'].tolist()
else:
    up_in_infected_pb_bknn = up_in_infected_pb_bknn_df.index.tolist()

# Optional: Preview number of genes
print(f"Genes upregulated in Infected (Pseudo-bulk BKNN):␣
↪{len(up_in_infected_pb_bknn)}")
```

```
Using None as control genes, passed at DeseqDataSet initialization
```

```
/tmp/ipykernel_3032317/3036063684.py:50: DeprecationWarning: ref_level is
deprecated and no longer has any effect. It will beremoved in a future release.
  dds = DeseqDataSet(
/tmp/ipykernel_3032317/3036063684.py:50: DeprecationWarning: design_factors is
deprecated and will soon be removed.Please consider providing a formulaic
formula using the design argumentinstead.
  dds = DeseqDataSet(
```

```
Fitting size factors…
… done in 0.01 seconds.

Fitting dispersions…
… done in 9.30 seconds.

Fitting dispersion trend curve…
/home1/bioinfo-22/.local/lib/python3.10/site-packages/pydeseq2/dds.py:804:
UserWarning: The dispersion trend curve fitting did not converge. Switching to a
mean-based dispersion trend.
  self._fit_parametric_dispersion_trend(vst)
… done in 1.06 seconds.

Fitting MAP dispersions…
… done in 16.73 seconds.

Fitting LFCs…
… done in 7.11 seconds.

Calculating cook's distance…
… done in 0.01 seconds.

Replacing 0 outlier genes.

Running Wald tests…

Log2 fold change & Wald test p-value: condition WT vs Infected
           baseMean  log2FoldChange     lfcSE      stat    pvalue      padj
Mrpl15    12.782605        0.292755  0.408983  0.715812  0.474108  0.939324
Gm37144    0.113688        1.187227  3.078263  0.385681  0.699733       NaN
Lypla1    11.095741        0.044092  0.454296  0.097055  0.922683  0.990502
Tcea1     30.973833        0.087179  0.269711  0.323233  0.746519  0.965406
Atp6v1h   10.284711        0.011811  0.477601  0.024731  0.980270  0.998023
…              …              …        …        …        …        …
Zp1        0.056601        2.082280  4.425346  0.470535  0.637973       NaN
Olfr1423   0.000000             NaN       NaN       NaN       NaN       NaN
Dmrt3      0.170588        0.602027  3.259224  0.184715  0.853453       NaN
Rbp4       0.000000             NaN       NaN       NaN       NaN       NaN
Gm6776     0.000000             NaN       NaN       NaN       NaN       NaN

[21216 rows x 6 columns]
Genes upregulated in Infected (Pseudo-bulk BKNN): 159

… done in 4.05 seconds.
```

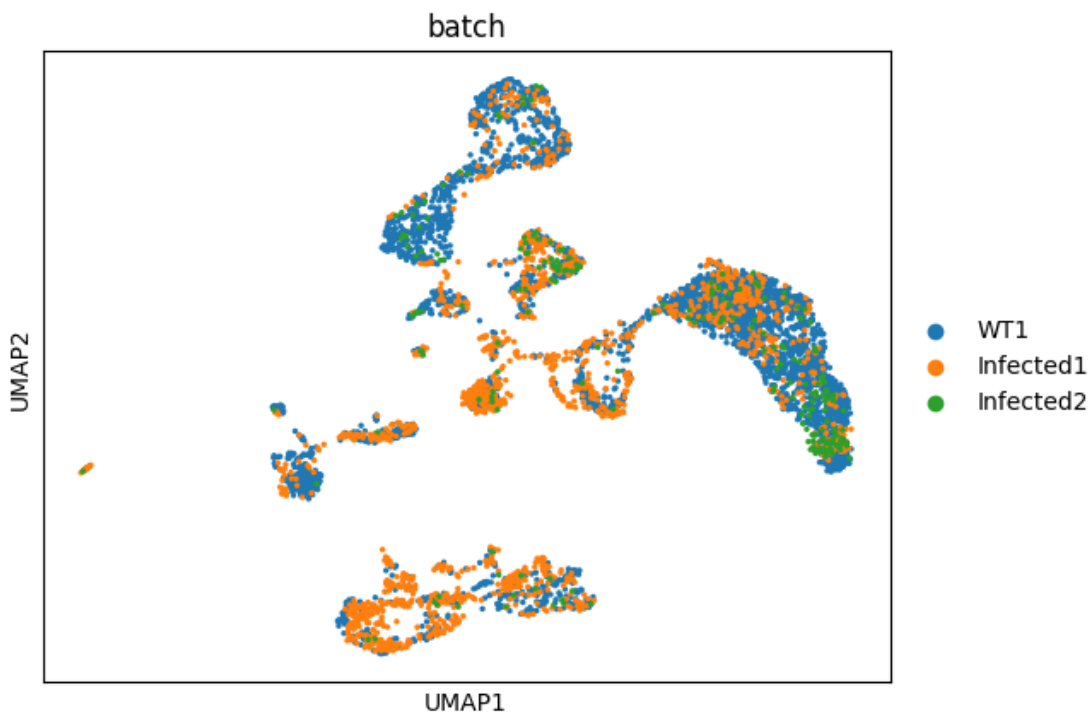
[27]:
```
# Harmony integration
# Run Harmony on PCA embedding
```

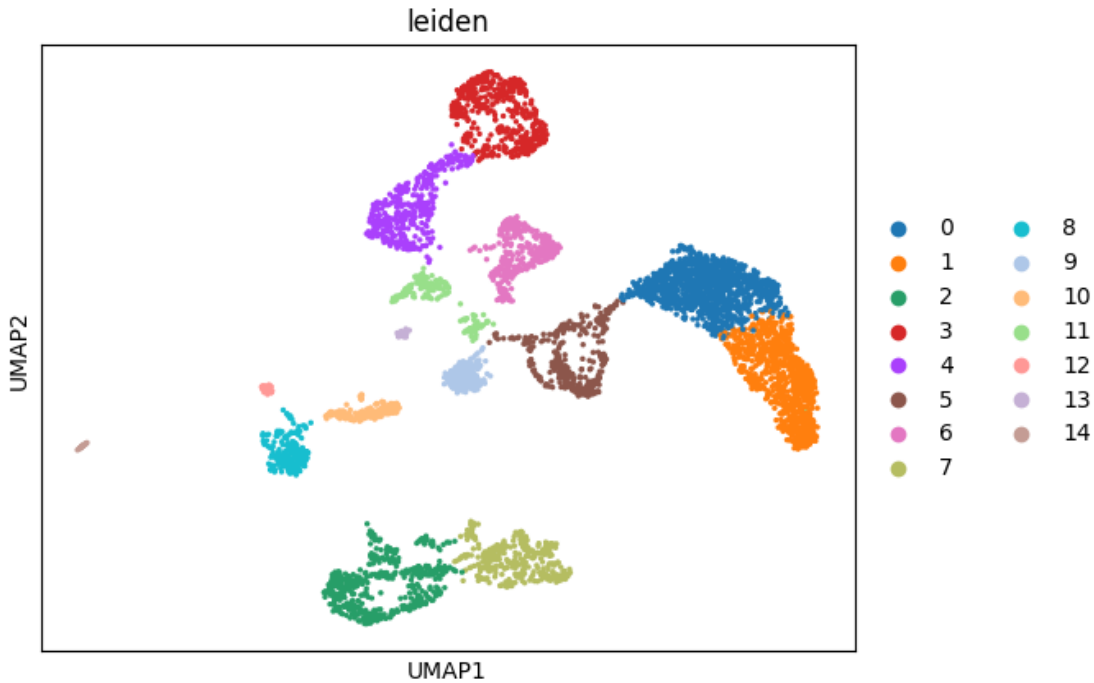
```
ho = hm.run_harmony(adata_concat.obsm['X_pca'], adata_concat.obs, 'batch')

# Store the corrected embeddings back into AnnData
adata_concat.obsm['X_pca_harmony'] = ho.Z_corr.T
```

```
2025-04-09 00:08:22,646 - harmonypy - INFO - Computing initial centroids with
sklearn.KMeans…
2025-04-09 00:10:08,776 - harmonypy - INFO - sklearn.KMeans initialization
complete.
2025-04-09 00:10:09,289 - harmonypy - INFO - Iteration 1 of 10
2025-04-09 00:10:25,776 - harmonypy - INFO - Iteration 2 of 10
2025-04-09 00:10:46,277 - harmonypy - INFO - Iteration 3 of 10
2025-04-09 00:11:04,275 - harmonypy - INFO - Iteration 4 of 10
2025-04-09 00:11:20,775 - harmonypy - INFO - Iteration 5 of 10
2025-04-09 00:11:32,179 - harmonypy - INFO - Iteration 6 of 10
2025-04-09 00:11:41,675 - harmonypy - INFO - Iteration 7 of 10
2025-04-09 00:11:52,875 - harmonypy - INFO - Converged after 7 iterations
```

[28]:
```
# Use Harmony-corrected PCA for neighbors and UMAP
sc.pp.neighbors(adata_concat, use_rep='X_pca_harmony')
sc.tl.umap(adata_concat)
sc.tl.leiden(adata_concat, resolution=0.5)
```

[29]:
```
sc.pl.umap(adata_concat, color=['batch'])       # Check batch mixing
sc.pl.umap(adata_concat, color=['leiden'])      # Check clustering
```



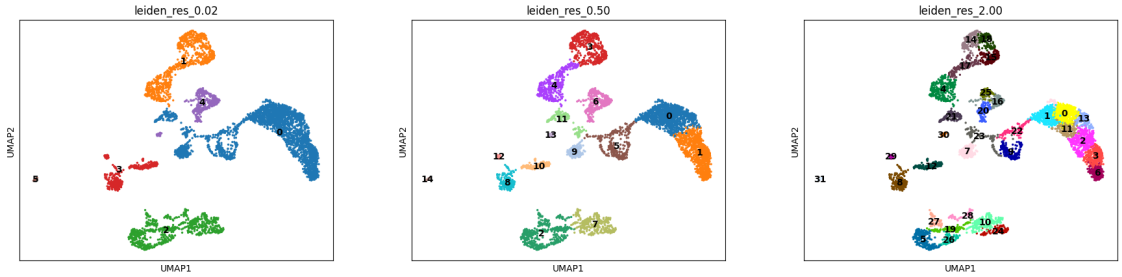


[30]:
```
# Count cells per Leiden cluster to identify top 3 largest clusters
cluster_sizes = adata_concat.obs['leiden'].value_counts().
 ↪sort_values(ascending=False)
print(cluster_sizes.head(3))
```

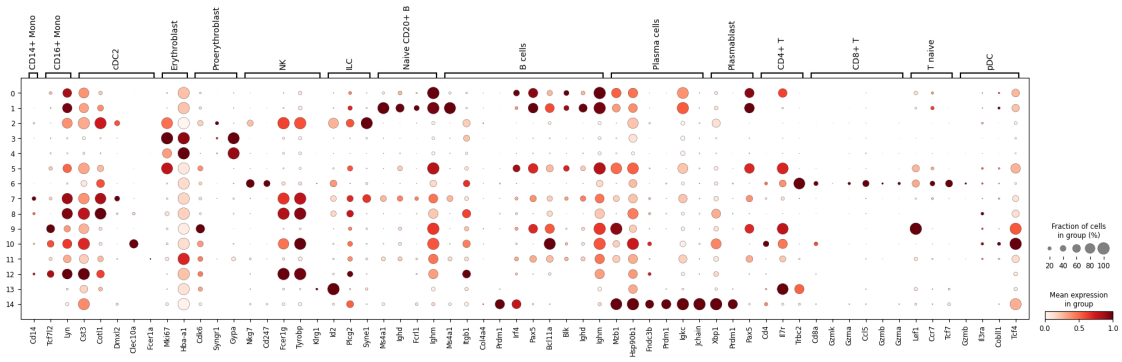
```
leiden
0    1156
1     862
2     472
Name: count, dtype: int64
```

[31]:
```
for res in [0.02, 0.5, 2.0]:
    sc.tl.leiden(
        adata_concat, key_added=f"leiden_res_{res:4.2f}", resolution=res
    )
```

[32]:
```
sc.pl.umap(
    adata_concat,
    color=["leiden_res_0.02", "leiden_res_0.50", "leiden_res_2.00"],
    legend_loc="on data",
)
```

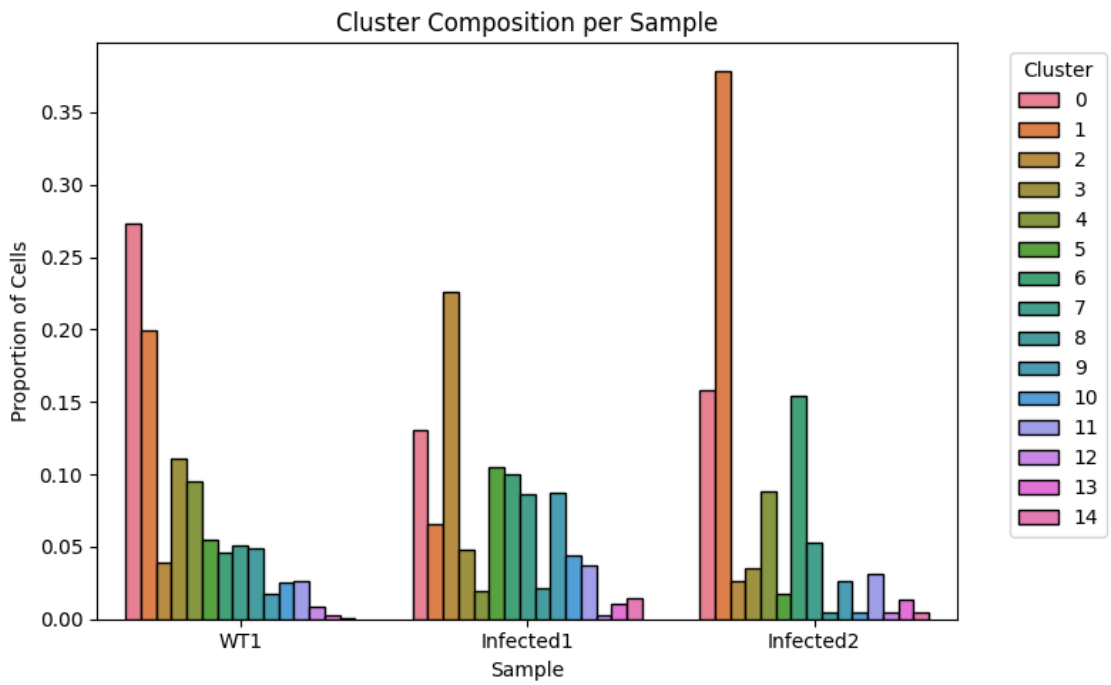


[33]:
```
# visualise what genes are elevated in every cluster under harmony integration
sc.pl.dotplot(adata_concat, marker_genes, groupby="leiden_res_0.50",␣
↪standard_scale="var")
```



[34]:
```
# Create a frequency table: clusters × samples
freq_table = pd.crosstab(adata_concat.obs['leiden'], adata_concat.obs['batch'])

# Convert counts to proportions (column-wise)
freq_table_prop = freq_table.div(freq_table.sum(axis=0), axis=1)

# Convert to long format for seaborn
freq_table_long = freq_table_prop.reset_index().melt(id_vars='leiden',␣
↪var_name='Sample', value_name='Proportion')

# Plot
plt.figure(figsize=(8, 5))
sns.barplot(data=freq_table_long, x='Sample', y='Proportion', hue='leiden',␣
↪edgecolor='black')

plt.xlabel("Sample")
plt.ylabel("Proportion of Cells")
plt.title("Cluster Composition per Sample")
plt.legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc='upper left')
```

```
plt.tight_layout()
plt.show()
```



[35]:
```
# Select cells from clusters 1 (Ms4a1 cluster) (harmony)
adata_b_like_harmony = adata_concat[adata_concat.obs['leiden'].isin(['1'])].
 ↪copy()
```

[36]:
```
# Create a unified condition column
adata_b_like_harmony.obs['condition'] = adata_b_like_harmony.obs['batch'].
 ↪replace({
    'WT1': 'WT',
    'Infected1': 'Infected',
    'Infected2': 'Infected'
})
```

[37]:
```
# Perform differential expression within the selected clusters
sc.tl.rank_genes_groups(
    adata_b_like_harmony,
    groupby='condition',
    groups=['WT'],
    reference='Infected',
    method='wilcoxon'
)
```

[38]:
```
# Convert to DataFrame and preview
de_results = sc.get.rank_genes_groups_df(adata_b_like_harmony, group='WT')
print(de_results.head(20))

# Save to CSV just in case
de_results.to_csv("Harmony_DE_WT_vs_Infected_in_Ms4a1_clusters_1_4.csv",␣
↪index=False)
```

```
        names     scores  logfoldchanges         pvals     pvals_adj
0      Hbb-bs  19.170624        2.084080  6.512258e-82  1.381641e-77
1      Hbb-bt  18.760836        2.640312  1.579002e-78  1.116670e-74
2      Hba-a1  18.141970        2.042275  1.486106e-73  7.882305e-70
3      S100a9  16.573227        2.066868  1.088245e-61  2.565356e-58
4      S100a8  15.302880        1.810495  7.314686e-53  1.034589e-49
5       H3f3a  14.564590        1.003117  4.718085e-48  4.170787e-45
6      Rps3a1  13.125811        0.623451  2.342636e-39  1.420039e-36
7      Hba-a2  12.490720        2.587001  8.389090e-36  4.237689e-33
8     mt-Cytb  11.884065        0.736482  1.432301e-32  6.201569e-30
9     mt-Atp6  11.234590        0.940795  2.757721e-29  9.916580e-27
10      Cd24a  10.892039        1.754892  1.257918e-27  4.170000e-25
11     Vpreb3  10.779472        2.109047  4.303472e-27  1.362723e-24
12      Cd79a  10.524399        0.803593  6.668545e-26  1.964998e-23
13      Actg1  10.288227        1.130386  7.962934e-25  2.252555e-22
14       Ftl1   9.972103        0.949667  2.019070e-23  5.099594e-21
15      Cd79b   9.723219        0.781774  2.400699e-22  5.659248e-20
16     mt-Nd4   9.521502        0.671801  1.706937e-21  3.936346e-19
17  Rps19-ps6   8.968537        6.158379  3.004786e-19  6.249956e-17
18       Fth1   8.855469        0.645475  8.333298e-19  1.637030e-16
19       Ebf1   8.627994        0.939544  6.243758e-18  1.141962e-15
```

[39]:
```
# Load the saved DE results
de_df_harmony = pd.read_csv("Harmony_DE_WT_vs_Infected_in_Ms4a1_clusters_1_4.
↪csv")

# Filter for genes upregulated in Infected: logFC < -0.5 and adjusted p < 0.01
up_in_infected_harmony_df = de_df_harmony[
    (de_df_harmony['logfoldchanges'] < -0.5) &
    (de_df_harmony['pvals_adj'] < 0.01)
]

# Extract gene names as a list
up_in_infected_harmony = up_in_infected_harmony_df['names'].tolist()

# Optional: Preview number of genes
print(f"Genes upregulated in Infected (Harmony): {len(up_in_infected_harmony)}")
```

```
Genes upregulated in Infected (Harmony): 310
```

[40]:
```
# Pseudo bulk for Harmony integrated data
# cluster 1 aka Ms4a1 cluster
adata_ms4a1 = adata_b_like_harmony[adata_b_like_harmony.obs['leiden_res_0.50']␣
↪== '1'].copy()
```

[41]:
```
# Combine Infected1 and Infected2 into 'Infected'
adata_ms4a1.obs['condition'] = adata_ms4a1.obs['batch'].replace({
    'WT1': 'WT',
    'Infected1': 'Infected',
    'Infected2': 'Infected'
})
```

[42]:
```
# Call the function that makes pseudo replicates (3 per condition)
adata_ms4a1 = assign_pseudoreps(adata_ms4a1, 'condition', n_reps=3)
```

[43]:
```
# Get expression matrix
expr_df = pd.DataFrame(
    adata_ms4a1.X.toarray() if hasattr(adata_ms4a1.X, "toarray") else␣
↪adata_ms4a1.X,
    index=adata_ms4a1.obs_names,
    columns=adata_ms4a1.var_names
)

# Combine metadata and expression
combined = pd.concat([adata_ms4a1.obs[['pseudorep', 'condition']], expr_df],␣
↪axis=1)

# Group by pseudoreplicate and sum ONLY expression columns
bulk_df = combined.groupby('pseudorep')[expr_df.columns].sum()

# Round and convert to int (DESeq2 needs integers)
counts = bulk_df.round().astype(int)

# Metadata: 1 row per pseudoreplicate
metadata = adata_ms4a1.obs[['pseudorep', 'condition']].drop_duplicates().
↪set_index('pseudorep')

# Build AnnData
adata_pb = ad.AnnData(X=counts.values)
adata_pb.var_names = counts.columns
adata_pb.obs = metadata
```

[44]:
```
# Run DESeq2
inference = DefaultInference(n_cpus=8)

dds = DeseqDataSet(
    adata=adata_pb,
```

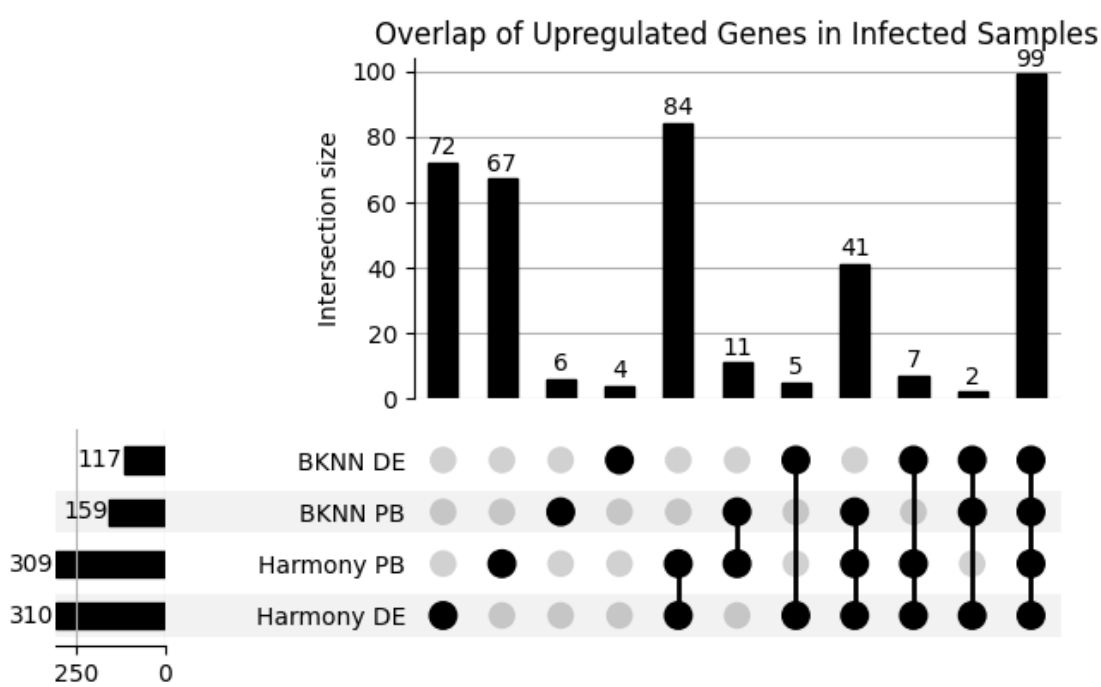
```
    design_factors='condition',
    ref_level=['condition', 'WT'],
    refit_cooks=True,
    inference=inference,
)
dds.deseq2()

stat_res = DeseqStats(
    dds,
    contrast=["condition", "WT", "Infected"],
    inference=inference,
)
stat_res.summary()

results_df = stat_res.results_df

# Filter for genes upregulated in Infected samples (padj < 0.01 and log2FC > 0.
↪5)
up_in_infected_pb_harmony_df = results_df[
    (results_df['padj'] < 0.01) &
    (results_df['log2FoldChange'] > 0.5)
]

# Extract gene names as a list
# Use column or index depending on your DESeq2 result structure
if 'gene' in up_in_infected_pb_harmony_df.columns:
    up_in_infected_pb_harmony = up_in_infected_pb_harmony_df['gene'].tolist()
else:
    up_in_infected_pb_harmony = up_in_infected_pb_harmony_df.index.tolist()

# Optional: show how many genes
print(f"Genes upregulated in Infected (Pseudo-bulk Harmony):␣
↪{len(up_in_infected_pb_harmony)}")
```

```
/tmp/ipykernel_3032317/1188611576.py:4: DeprecationWarning: ref_level is
deprecated and no longer has any effect. It will beremoved in a future release.
  dds = DeseqDataSet(
/tmp/ipykernel_3032317/1188611576.py:4: DeprecationWarning: design_factors is
deprecated and will soon be removed.Please consider providing a formulaic
formula using the design argumentinstead.
  dds = DeseqDataSet(
Fitting size factors…
… done in 0.01 seconds.


Using None as control genes, passed at DeseqDataSet initialization

Fitting dispersions…
```

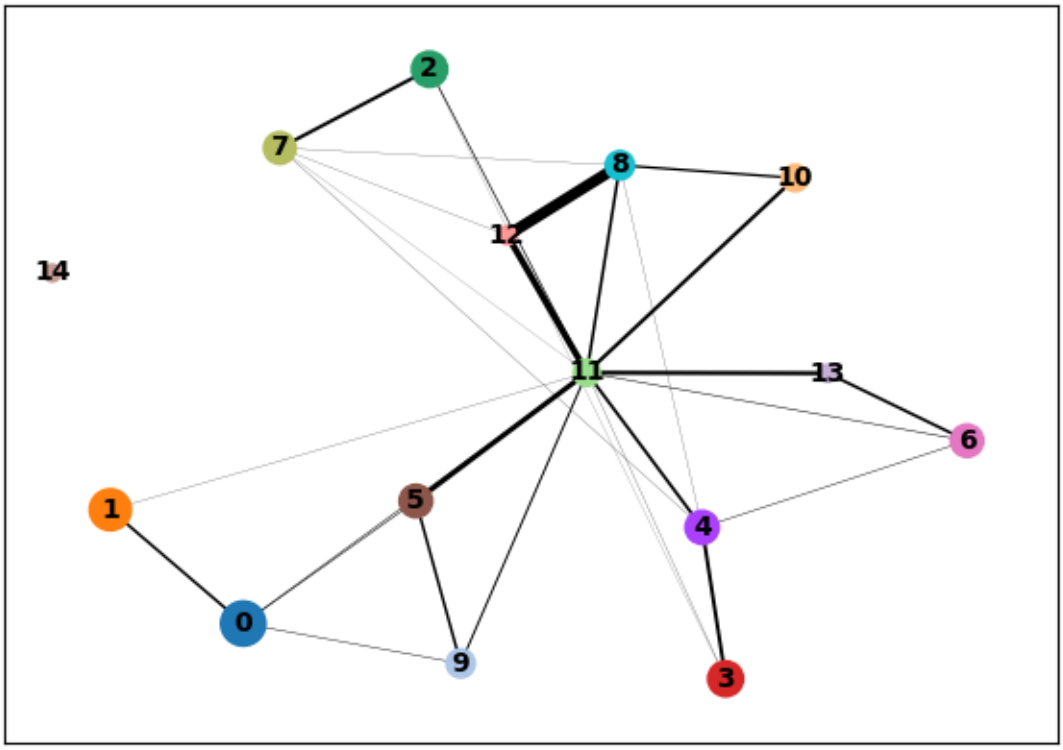
```
… done in 8.80 seconds.

Fitting dispersion trend curve…
/home1/bioinfo-22/.local/lib/python3.10/site-packages/pydeseq2/dds.py:804:
UserWarning: The dispersion trend curve fitting did not converge. Switching to a
mean-based dispersion trend.
  self._fit_parametric_dispersion_trend(vst)
… done in 0.67 seconds.

Fitting MAP dispersions…
… done in 17.11 seconds.

Fitting LFCs…
… done in 7.65 seconds.

Calculating cook's distance…
… done in 0.01 seconds.

Replacing 0 outlier genes.

Running Wald tests…
… done in 4.04 seconds.


Log2 fold change & Wald test p-value: condition WT vs Infected
           baseMean  log2FoldChange     lfcSE      stat    pvalue      padj
Mrpl15    25.685346        0.348895  0.285631  1.221491  0.221900  0.632829
Gm37144    0.086097        0.907872  4.425373  0.205152  0.837454       NaN
Lypla1    17.872485       -0.156706  0.476649 -0.328765  0.742333  0.932184
Tcea1     57.464357        0.127904  0.191277  0.668687  0.503695  0.838204
Atp6v1h   16.377346       -0.014241  0.354901 -0.040128  0.967991  0.995012
…              …              …        …        …        …        …
Zp1        0.000000             NaN       NaN       NaN       NaN       NaN
Olfr1423   0.000000             NaN       NaN       NaN       NaN       NaN
Dmrt3      0.255863       -0.569899  2.788374 -0.204384  0.838053       NaN
Rbp4       0.000000             NaN       NaN       NaN       NaN       NaN
Gm6776     0.000000             NaN       NaN       NaN       NaN       NaN

[21216 rows x 6 columns]
Genes upregulated in Infected (Pseudo-bulk Harmony): 309
```

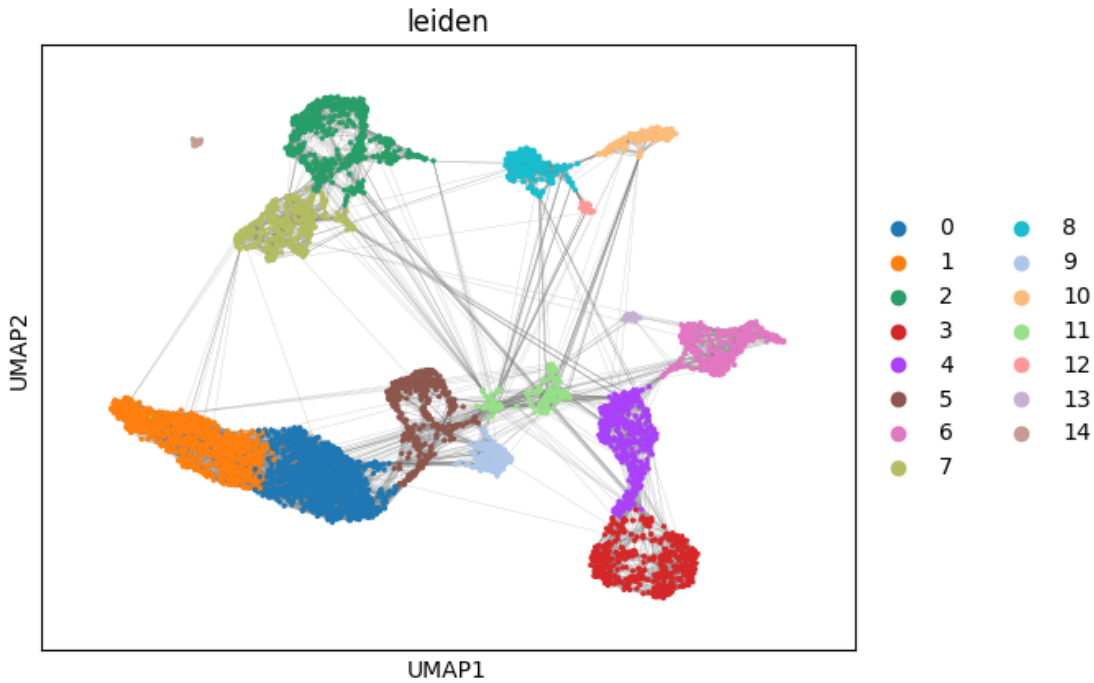
[46]:
```
# Generate a summary figure/plot to compare the four different lists of genes
#UpSet plot (to show upregulated genes in four different methods and shared␣
 ↪genes in the methods)

# Define the sets
set1 = set(up_in_infected_bknn)
set2 = set(up_in_infected_pb_bknn)
```

```python
set3 = set(up_in_infected_harmony)
set4 = set(up_in_infected_pb_harmony)

# Build a list of memberships for upsetplot
all_genes = set1 | set2 | set3 | set4
memberships = []
for gene in all_genes:
    present_in = []
    if gene in set1:
        present_in.append("BKNN DE")
    if gene in set2:
        present_in.append("BKNN PB")
    if gene in set3:
        present_in.append("Harmony DE")
    if gene in set4:
        present_in.append("Harmony PB")
    memberships.append(present_in)

# Create data and plot
data = from_memberships(memberships)
UpSet(data, subset_size='count', show_counts=True).plot()
plt.title("Overlap of Upregulated Genes in Infected Samples")
plt.show()
```



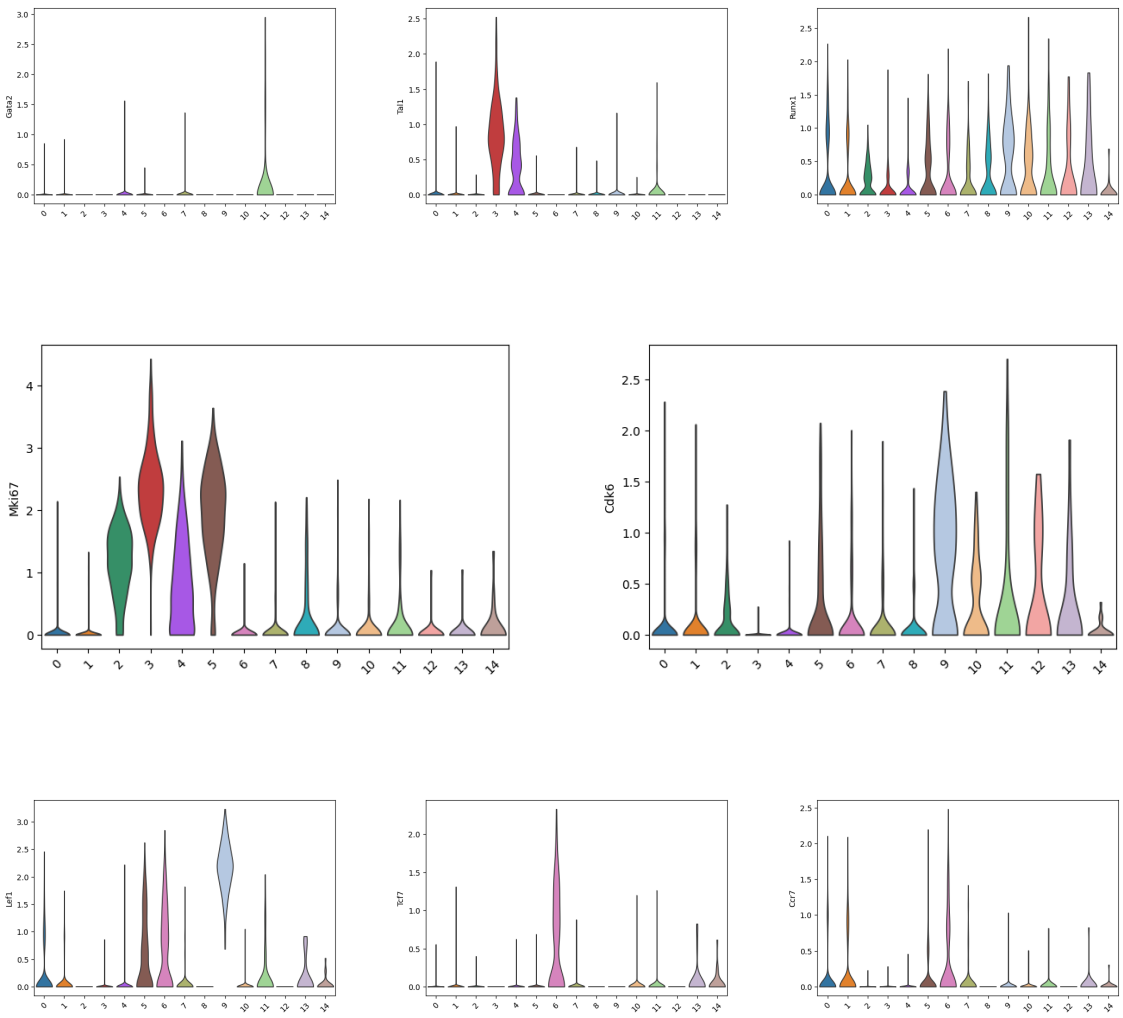
[47]:
```
# pseudo time PAGA
# Choose a clustering result to use as groups (e.g., 'leiden')
sc.tl.paga(adata_concat, groups='leiden')

# Visualize the abstracted graph on top of UMAP
sc.pl.paga(adata_concat, plot=True)

# Embed PAGA into UMAP
sc.tl.umap(adata_concat, init_pos='paga')
sc.pl.umap(adata_concat, color=['leiden'], edges=True)
```





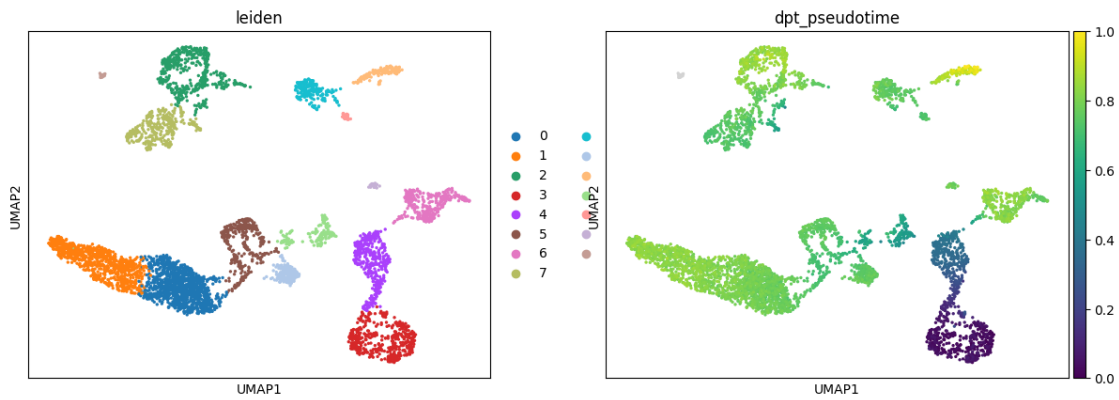
[48]:
```
# assume Ms4a1 cluster is the end
#---------------------------------
# look for potential starter cluster (Hematopoietic stem/progenitor cells with␣
↪genes Gata2, Tal1, Runx1)
sc.pl.violin(
    adata_concat,
    keys=["Gata2", "Tal1", "Runx1"],
    groupby="leiden_res_0.50",
    stripplot=False,
    rotation=45,
    multi_panel=True
)
# clusters 3 and 4 esp 3 seems like hematopoietic stem/progenitor
#-------------------------------------------------------------------
# look for potential starter cluster (Proliferative progenitors with genes␣
↪Mki67, Cdk6)
sc.pl.violin(
    adata_concat,
    keys=["Mki67", "Cdk6"],
    groupby="leiden_res_0.50",
    stripplot=False,
    rotation=45,
    multi_panel=True
)
```

```
# cluster 3 has seems like proliferative progenitors
#-----------------------------------------------------
# look for potential starter cluster ( Naive T cells (early))
sc.pl.violin(
    adata_concat,
    keys=["Lef1", "Tcf7", "Ccr7"],
    groupby="leiden_res_0.50",
    stripplot=False,
    rotation=45,
    multi_panel=True
)
# cluster 9 seems like early naive T cells
```

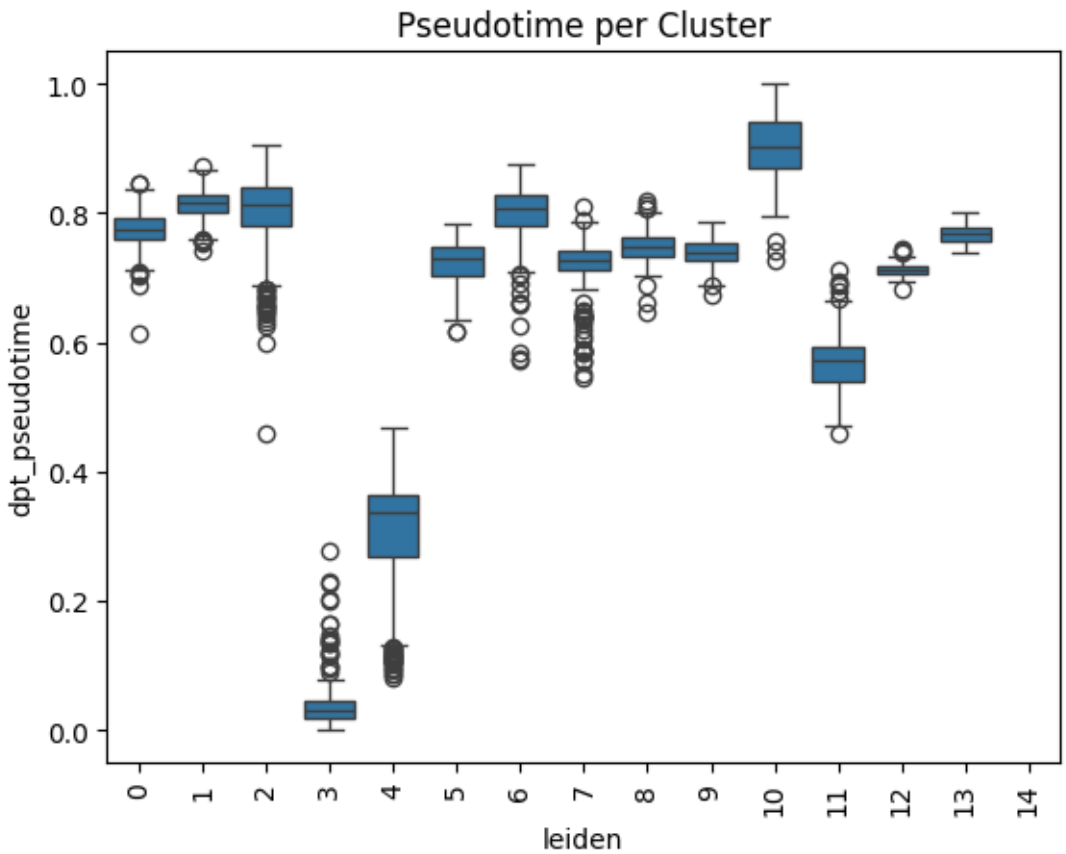


[49]:
```
# From above violin plots: assume cluster 3 is the start

# Set the root cell
```

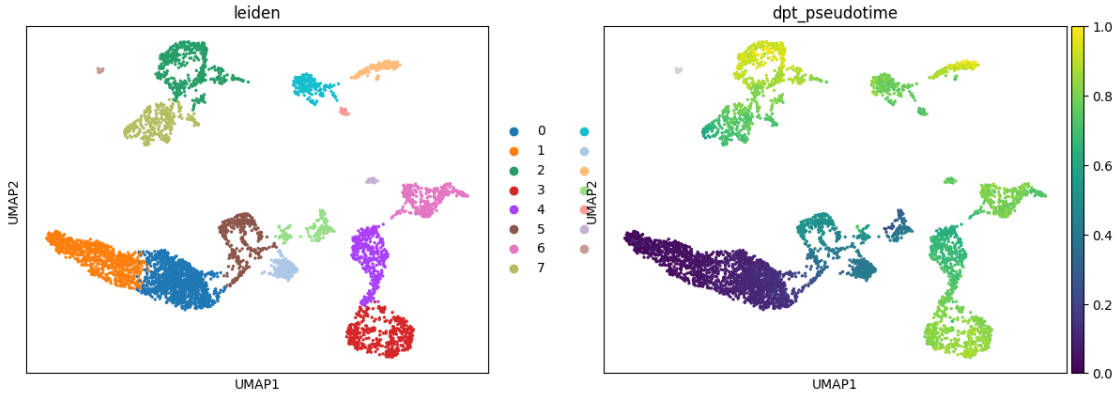
```
root_cell = adata_concat.obs[adata_concat.obs['leiden'] == '3'].index[0]
adata_concat.uns['iroot'] = adata_concat.obs_names.get_loc(root_cell)

# compute diffusion map + pseudotime
sc.tl.diffmap(adata_concat)
sc.tl.dpt(adata_concat)

# Visualize
sc.pl.umap(adata_concat, color=['leiden', 'dpt_pseudotime'],␣
↪color_map='viridis')
```



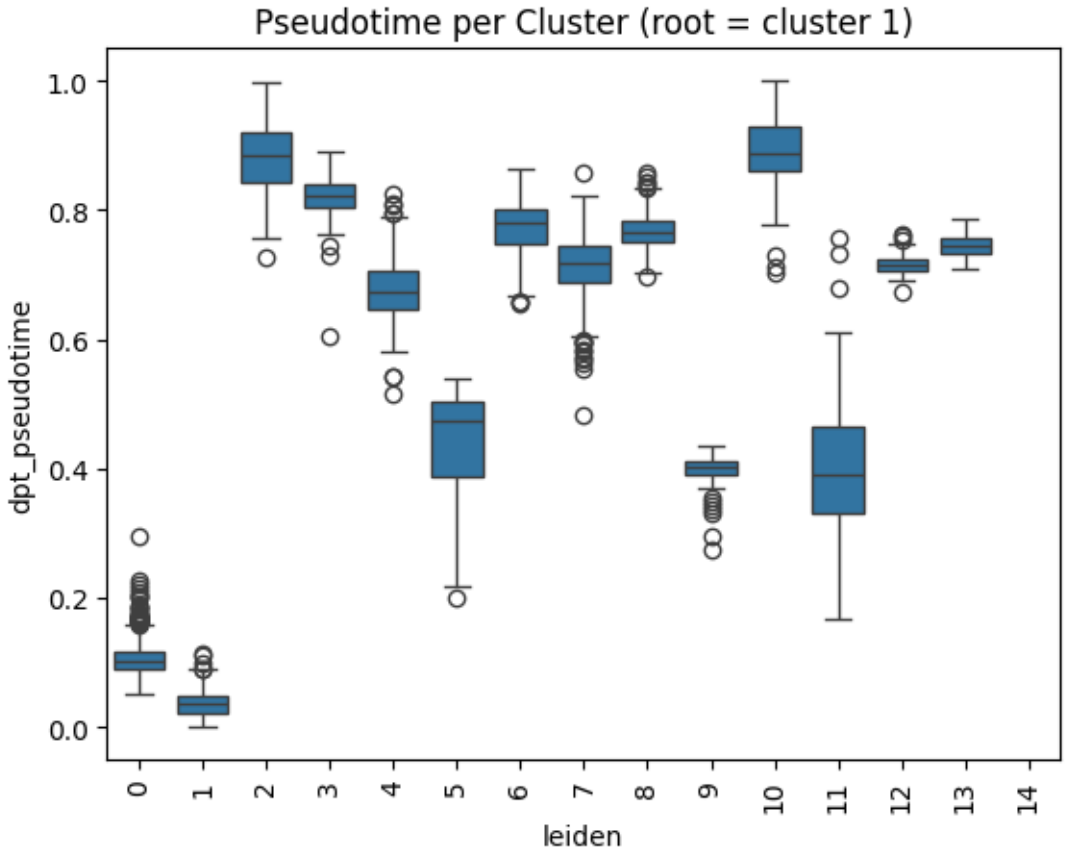
[50]:
```
# cluster 10 seems like the end point, rather than cluster 1(Ms4a1 cluster)
sns.boxplot(x=adata_concat.obs['leiden'], y=adata_concat.obs['dpt_pseudotime'])
plt.xticks(rotation=90)
plt.title("Pseudotime per Cluster")
plt.show()
```



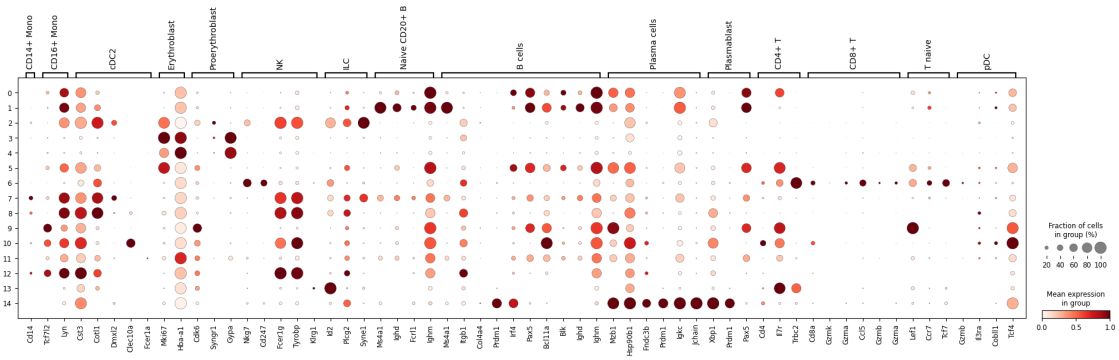
[51]:
```
# assume Ms4a1 cluster is the start
# Set cluster 1 (Ms4a1) as the root
root_cell = adata_concat.obs[adata_concat.obs['leiden'] == '1'].index[0]
adata_concat.uns['iroot'] = adata_concat.obs_names.get_loc(root_cell)

# Recompute diffusion map and DPT
sc.tl.diffmap(adata_concat)
sc.tl.dpt(adata_concat)

# Visualize
sc.pl.umap(adata_concat, color=['leiden', 'dpt_pseudotime'],␣
↪color_map='viridis')
```



[52]:
```
# cluster 10 seems like the end point again even when we set start as Ms4a1␣
↪cluster (cluster1)
sns.boxplot(x=adata_concat.obs['leiden'], y=adata_concat.obs['dpt_pseudotime'])
plt.xticks(rotation=90)
plt.title("Pseudotime per Cluster (root = cluster 1)")
plt.show()
```



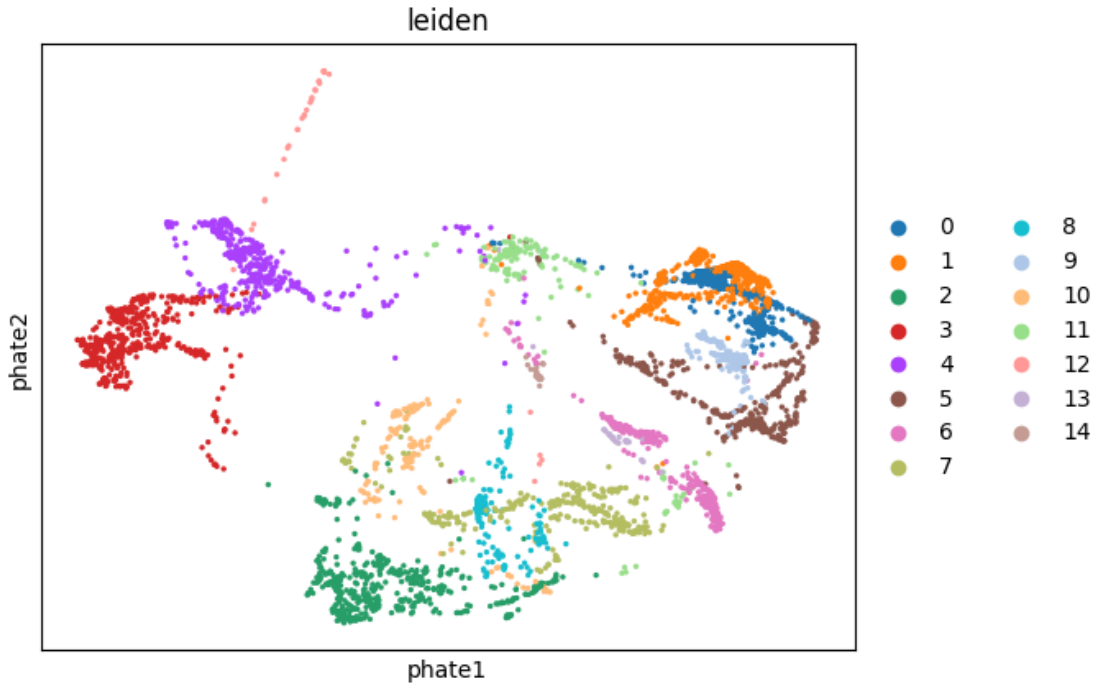
[53]:
```
# cluster 10 seems to be mix of effector cells like NK, plasmablasts, T
sc.pl.dotplot(adata_concat, marker_genes, groupby="leiden_res_0.50",␣
↪standard_scale="var")
```



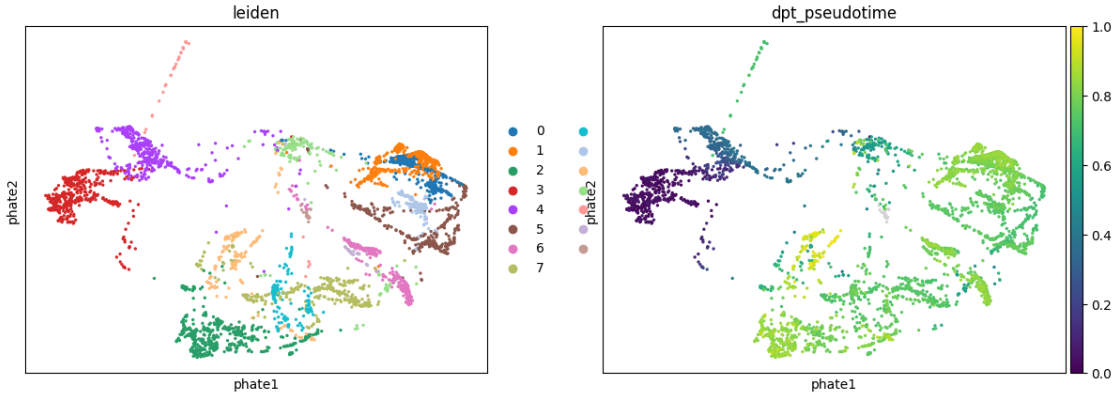
[54]:
```
# PHATE
phate_op = phate.PHATE()
adata_concat.obsm['X_phate'] = phate_op.fit_transform(adata_concat.X)

# Plot PHATE colored by cluster or pseudotime
sc.pl.embedding(adata_concat, basis='phate', color='leiden')
```

```
Calculating PHATE…
  Running PHATE on 5099 observations and 21216 variables.
  Calculating graph and diffusion operator…
    Calculating PCA…
    Calculated PCA in 80.27 seconds.
    Calculating KNN search…
    Calculated KNN search in 1.83 seconds.
    Calculating affinities…
    Calculated affinities in 0.16 seconds.
  Calculated graph and diffusion operator in 82.36 seconds.
  Calculating landmark operator…
    Calculating SVD…
    Calculated SVD in 14.51 seconds.
    Calculating KMeans…
    Calculated KMeans in 115.39 seconds.
  Calculated landmark operator in 130.83 seconds.
  Calculating optimal t…
    Automatically selected t = 27
  Calculated optimal t in 126.09 seconds.
  Calculating diffusion potential…
  Calculated diffusion potential in 9.69 seconds.
  Calculating metric MDS…
```

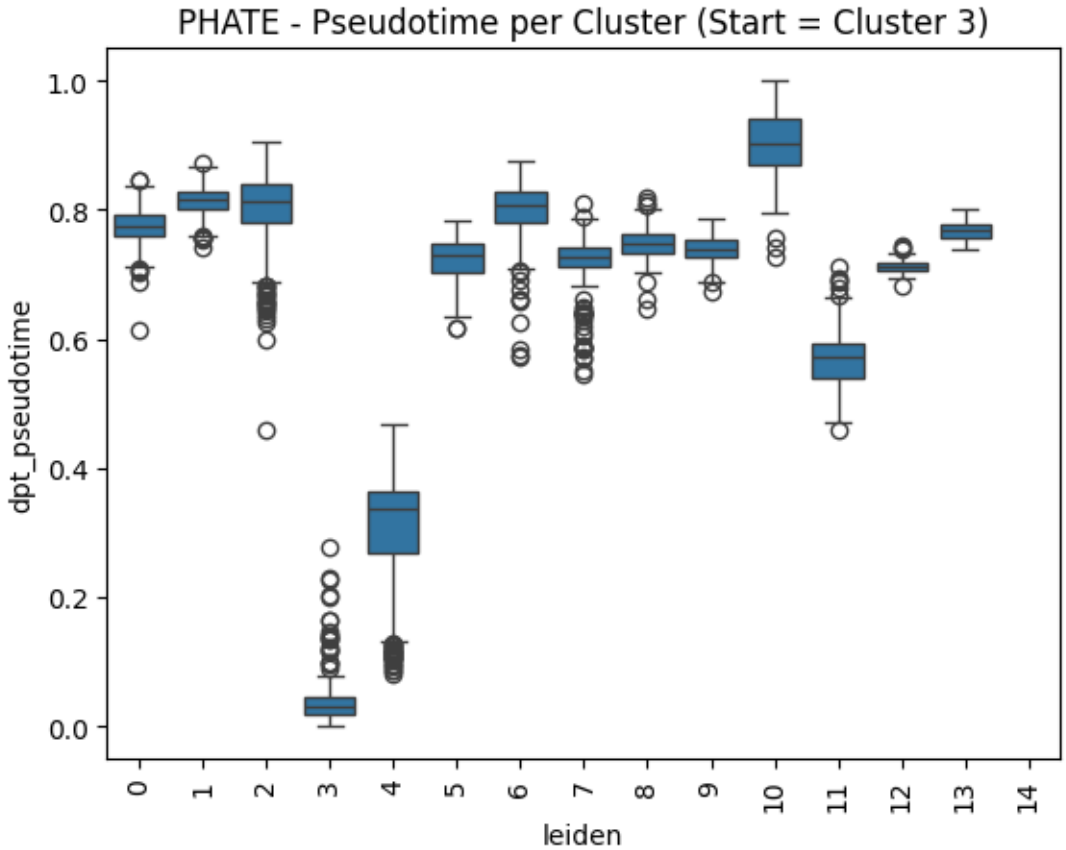
```
  Calculated metric MDS in 7.07 seconds.
Calculated PHATE in 356.06 seconds.
```



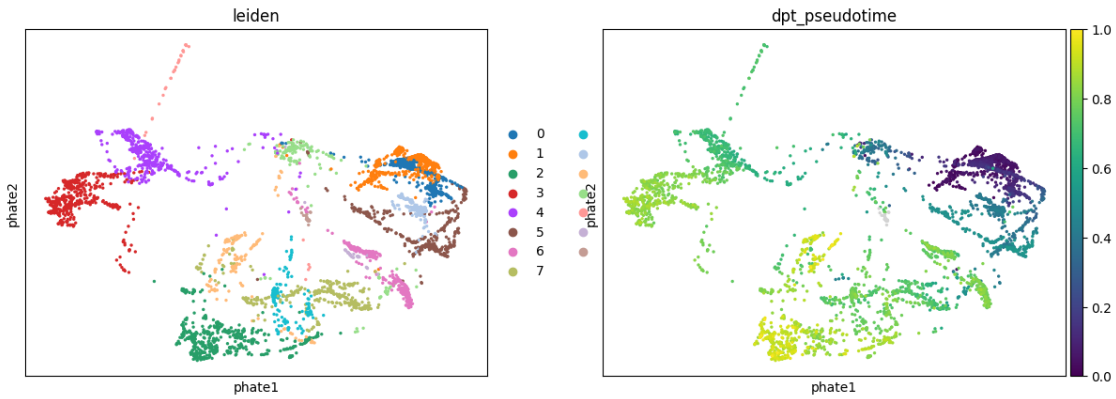
[55]:
```
# assume Ms4a1 cluster (cluster1) as end
# Set root as cluster 3 as proven earlier by violin plots to be progenitor
root_cell = adata_concat.obs[adata_concat.obs['leiden'] == '3'].index[0]
adata_concat.uns['iroot'] = adata_concat.obs_names.get_loc(root_cell)

# Recompute pseudotime
sc.tl.diffmap(adata_concat)
sc.tl.dpt(adata_concat)

# Visualize
sc.pl.embedding(adata_concat, basis='phate', color=['leiden',␣
↪'dpt_pseudotime'], color_map='viridis')
```



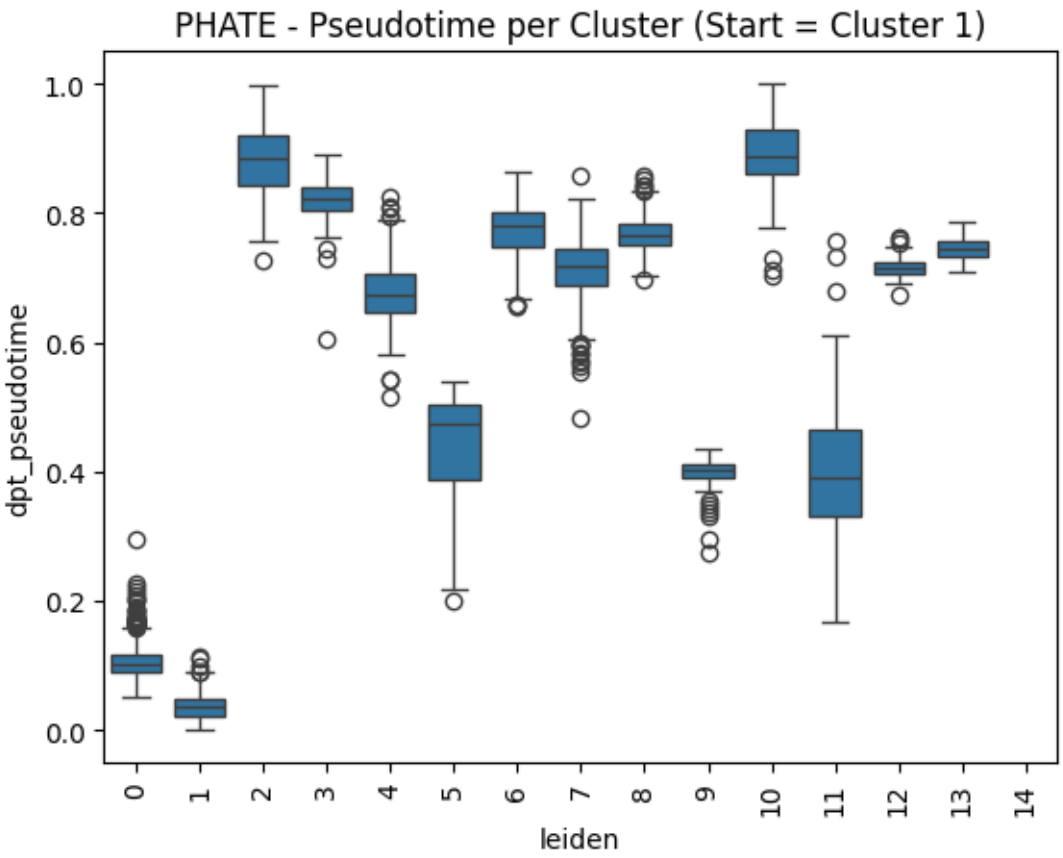
[56]:
```
#show pseudotime per cluster
sns.boxplot(x=adata_concat.obs['leiden'], y=adata_concat.obs['dpt_pseudotime'])
plt.xticks(rotation=90)
plt.title("PHATE - Pseudotime per Cluster (Start = Cluster 3)")
plt.show()
#again, cluster 10 seems to be the terminal
```



[57]:
```
# assume cluster 1 (Ms4a1) as the start
# Set root as cluster 1 (Ms4a1)
root_cell = adata_concat.obs[adata_concat.obs['leiden'] == '1'].index[0]
adata_concat.uns['iroot'] = adata_concat.obs_names.get_loc(root_cell)

# Re-run pseudotime
sc.tl.diffmap(adata_concat)
sc.tl.dpt(adata_concat)

# Visualize on PHATE
sc.pl.embedding(adata_concat, basis='phate', color=['leiden',␣
↪'dpt_pseudotime'], color_map='viridis')
```



[58]:
```
#show pseudotime per cluster
sns.boxplot(x=adata_concat.obs['leiden'], y=adata_concat.obs['dpt_pseudotime'])
plt.xticks(rotation=90)
plt.title("PHATE - Pseudotime per Cluster (Start = Cluster 1)")
plt.show()
#again, cluster 10 seems to be the terminal
```


PHATE - Pseudotime per Cluster (Start = Cluster 1)
dpt_pseudotime
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
leiden